



COMP 412
FALL 2012

Bottom-Up Parsing, Part II
— LR(1) Parsers —
Comp 412

Copyright 2012, Keith D. Cooper & Linda Torczon, all rights reserved.

Students enrolled in Comp 412 at Rice University have explicit permission to make copies of these materials for their personal use.

Faculty from other educational institutions may use these materials for nonprofit educational purposes, provided this copyright notice is preserved.



LR Parsers — A Roadmap

- Last Lecture
 - Table-driven LL(1) parsers (see Lab 2)
 - Bottom-up, shift-reduce parsing and handles (examples)
- Today
 - More on handles (theory)
 - Skeleton LR(1) parser & its operation
 - (*time permitting*) notation & concepts for the LR(1) Table Construction
- Monday
 - LR(1) Table construction



Handles

Consider the example from last lecture

Sentential Form

derivation ↓	Goal				
	Expr				
	Expr	—	Term		
	Expr	—	Term	*	Factor
	Expr	—	Term	*	<id,y>
	Expr	—	Factor	*	<id,y>
	Expr	—	<num,2>	*	<id,y>
	Term	—	<num,2>	*	<id,y>
	Factor	—	<num,2>	*	<id,y>
	<id,x>	—	<num,2>	*	<id,y>
					↑ parse

Unambiguous grammar implies unique rightmost derivation

- At each step, we have one step that leads to $x - 2 * y$
- Any other choice leads to another distinct expression

A bottom-up parse reverses the rightmost derivation

- It has a unique reduction at each step
- The key is finding the reduction at each step that leads to the derivation



Handles

Consider the example from last lecture

<i>Sentential Form</i>				<i>Reduction</i>
<i>Goal</i>				
<i>Expr</i>				$Goal \rightarrow Expr$
<i>Expr</i>	—	<i>Term</i>		$Expr \rightarrow Expr - Term$
<i>Expr</i>	—	<i>Term</i>	* <i>Factor</i>	$Term \rightarrow Term * Factor$
<i>Expr</i>	—	<i>Term</i>	* <i><id,y></i>	$Factor \rightarrow \underline{id}$
<i>Expr</i>	—	<i>Factor</i>	* <i><id,y></i>	$Term \rightarrow Factor$
<i>Expr</i>	—	<i><num,2></i>	* <i><id,y></i>	$Factor \rightarrow \underline{num}$
<i>Term</i>	—	<i><num,2></i>	* <i><id,y></i>	$Expr \rightarrow Term$
<i>Factor</i>	—	<i><num,2></i>	* <i><id,y></i>	$Term \rightarrow Factor$
<i><id,x></i>	—	<i><num,2></i>	* <i><id,y></i>	$Factor \rightarrow \underline{id}$



Handles

The bottom-up, shift-reduce parser operates by repeatedly finding a substring β of the tree's upper frontier that matches some production $A \rightarrow \beta$ and replacing β with A .

For the parse to succeed, it must find the β that occurs as the next step of the parse (or the previous step of the derivation).

Formally,

A **handle** of a right-sentential form γ is a pair $\langle A \rightarrow \beta, k \rangle$ where $A \rightarrow \beta \in P$ and k is the position in γ of β 's rightmost symbol.

If $\langle A \rightarrow \beta, k \rangle$ is a handle, then replacing β at k with A produces the right sentential form from which γ is derived in the rightmost derivation.

Because γ is a right-sentential form, the substring to the right of a handle contains **only terminal symbols**

$\Rightarrow$ the parser doesn't need to scan (*much*) past the handle



Handles

Consider the example from last lecture

Sentential Form

Goal

Expr

Expr — *Term*

Expr — *Term* * *Factor*

Expr — *Term* * $\langle \text{id}, y \rangle$

Expr — *Factor* * $\langle \text{id}, y \rangle$

Expr — $\langle \text{num}, 2 \rangle$ * $\langle \text{id}, y \rangle$

Term — $\langle \text{num}, 2 \rangle$ * $\langle \text{id}, y \rangle$

Factor — $\langle \text{num}, 2 \rangle$ * $\langle \text{id}, y \rangle$

$\langle \text{id}, x \rangle$ — $\langle \text{num}, 2 \rangle$ * $\langle \text{id}, y \rangle$

Now, look at the sentential forms in the example

- They have a specific form
- $NT^* (NT | T)^* T^*$
- Handles are found in the $(NT | T)^*$ portion
 - Track right end of region
 - Search left from there
 - Finite set of rhs strings

We know that each step has a unique reduction

That reduction is the handle



From Handles to Parsers

The sentential forms in the derivation have the form:

$$NT^* (NT | T)^* T^*$$

- They are the upper fringe of partially complete syntax tree
- The suffix consisting of T^* is, at each step, the unread input

The shift-reduce parser operates by:

- Keeping the portion $NT^* (NT | T)^*$ on a stack
 - Leftmost symbol at bottom of stack, rightmost at stack top
- Handles always appear with right end at the top of stack
 - *The border between $NT^* (NT | T)^*$ and T^* is the critical spot*
- Searching for handles from stack top to stack bottom
- If search fails, shift another terminal onto stack



Handles Are Unique

Theorem:

*If G is unambiguous, then every right-sentential form has a **unique** handle.*

Recall: Right sentential form is a string that appears as one step in a rightmost derivation.

Sketch of Proof:

- 1 G is unambiguous $\Rightarrow$ rightmost derivation is unique
- 2 $\Rightarrow$ a unique production $A \rightarrow \beta$ applied to derive γ_i from γ_{i-1}
- 3 $\Rightarrow$ a unique position k at which $A \rightarrow \beta$ is applied
- 4 $\Rightarrow$ a unique handle $\langle A \rightarrow \beta, k \rangle$

This all follows from the definitions

If we can find the handles, we can build a derivation!

*The handle always appears with its right end at the stack top.
→ We can make the handles relative to the stack top, which makes the "position" implicit. Now the handle is just the "right" RHS at stack top.*

Recursive invocation of the DFA to look for the "right" RHS



An Important Lesson about Handles

Simply looking for right hand sides that match strings is not good enough

Critical Question: How can we know when we have found a handle without generating lots of different derivations?

- **Answer:** We use left context, encoded in the sentential form, left context encoded in a "parser state", and a lookahead at the next word in the input. (Formally, 1 word beyond the handle.)
- Parser states are derived by reachability analysis on grammar that resembles the subset construction from Chapter 2.
- We build all of this knowledge into a handle-recognizing DFA

The additional left context is precisely the reason that LR(1) grammars express a superset of the languages that can be expressed as LL(1) grammars

LR(1) Parsers



- LR(1) parsers are table-driven, shift-reduce parsers that use a limited right context (1 token) for handle recognition
- The class of grammars that these parsers recognize is called the set of LR(1) grammars

Informal definition:

A grammar is LR(1) if, given a rightmost derivation

$$S \Rightarrow \gamma_0 \Rightarrow \gamma_1 \Rightarrow \gamma_2 \Rightarrow \dots \Rightarrow \gamma_{n-1} \Rightarrow \gamma_n \Rightarrow \text{sentence}$$

We can

1. *isolate the handle of each right-sentential form γ_i , and*
2. *determine the production by which to reduce,*

by scanning γ_i from *left-to-right*, going at most 1 symbol beyond the right end of the handle of γ_i

LR(1) means *left-to-right scan* of the input, *rightmost derivation* (in reverse), and *1 word of lookahead*.

I always find this definition to be curiously unsatisfying because it isn't an operational definition.

Bottom-up Parser

from last lecture



A conceptual *shift-reduce parser*:

```
push INVALID
word ← NextWord( )
repeat until (top of stack = Goal and word = EOF)
  if the top of the stack is a handle  $A \rightarrow \beta$ 
    then // reduce  $\beta$  to  $A$ 
      pop  $|\beta|$  symbols off the stack
      push  $A$  onto the stack
  else if (word  $\neq$  EOF)
    then // shift
      push word
      word ← NextWord( )
  else // need to shift, but out of input
    report an error
```

Shift-reduce parsers have four kinds of actions:

Shift: next word is moved from input to stack

Reduce: handle is at TOS
pop rhs of handle
push lhs of handle

Accept: stop & report success

Error: report an error

Shift & Accept are $O(1)$

Reduce is $O(|\text{rhs}|)$

Key insight: the parser shifts until a handle appears at TOS



Actual LR(1) Skeleton Parser

```
stack.push(INVALID);
stack.push( $s_0$ ); // initial state
word = scanner.next_word();
loop forever {
    s = stack.top();
    if ( ACTION[s,word] == "reduce  $A \rightarrow \beta$ " ) then {
        stack.popnum( $2 * |\beta|$ ); // pop  $2 * |\beta|$  symbols
        s = stack.top();
        stack.push(A); // push A
        stack.push(GOTO[s,A]); // push next state
    }
    else if ( ACTION[s,word] == "shift  $s_i$ " ) then {
        stack.push(word); stack.push( $s_i$ );
        word ← scanner.next_word();
    }
    else if ( ACTION[s,word] == "accept"
              & word == EOF )
        then break;
    else throw a syntax error;
}
report success;
```

The skeleton LR(1) parser

- follows basic scheme for shift-reduce parsing from last slide
- relies on a stack & a scanner
- Stacks <symbol, state> pairs
- uses two tables, called ACTION & GOTO
- shifts |words| times
- reduces |derivation| times
- accepts at most once
- detects errors by failure of the other three cases, which is a failure to find a handle

The Parentheses Language



Language of balanced parentheses

- Beyond power of REs
- Exhibits role of context in LR(1) parsing

0	Goal	→	List
1	List	→	List Pair
2			Pair
3	Pair	→	(Pair)
4			()



The Parentheses Language

ACTION TABLE				GOTO TABLE		
State	EOF	(	)	State	List	Pair
0		S 3		0	1	2
1	<i>acc</i>	S 3		1		4
2	R 2	R 2		2		
3		S 6	S 7	3		5
4	R 1	R 1		4		
5			S 8	5		
6		S 6	S 10	6		9
7	R 4	R 4		7		
8	R 3	R 3		8		
9			S 11	9		
10			R 4	10		
11			R 3	11		

0	Goal	→	List
1	List	→	List Pair
2			Pair
3	Pair	→	(Pair)
4			()



The Parentheses Language

State	Lookahead	Stack	Handle	Action
—	(	\$ 0	<i>—none—</i>	—
0	(	\$ 0	<i>—none—</i>	<i>shift 3</i>
3	)	\$ 0 (3	<i>—none—</i>	<i>shift 7</i>
7	eof	\$ 0 (3) 7	()	<i>reduce 4</i>
2	eof	\$ 0 Pair 2	Pair	<i>reduce 2</i>
1	eof	\$ 0 List 1	List	<i>accept</i>

Parsing "()"

0	Goal	→	List
1	List	→	List Pair
2			Pair
3	Pair	→	(Pair)
4			()

The Parentheses Language



State	L'ahead	Stack	Handle	Action
—	(	\$ 0	—none—	—
0	(	\$ 0	—none—	shift 3
3	(	\$ 0 (3	—none—	shift 6
6	)	\$ 0 (3 (6	—none—	shift 10
10	)	\$ 0 (3 (6) 10	()	reduce 4
5	)	\$ 0 (3 Pair 5	—none—	shift 8
8	(	\$ 0 (3 Pair 5) 8	(Pair)	reduce 3
2	(	\$ 0 Pair 2	Pair	reduce 2
1	(	\$ 0 List 1	—none—	shift 3
3	)	\$ 0 List 1 (3	—none—	shift 7
7	eof	\$ 0 List 1 (3) 7	()	reduce 4
4	eof	\$ 0 List 1 Pair 4	List Pair	reduce 1
1	eof	\$ 0 List 1	List	accept

Parsing
"(()) ()"

Let's look at
how the parser
reduces "()"

0	Goal	→	List
1	List	→	List Pair
2			Pair
3	Pair	→	(Pair)
4			()



The Parentheses Language

State	Lookahead	Stack	Handle	Action
—	(	\$ 0	—none—	—
0	(	\$ 0	—none—	shift 3
3	)	\$ 0 (3	—none—	shift 7
7	eof	\$ 0 (3) 7	()	reduce 4
2	eof	\$ 0 Pair 2	Pair	reduce 2
1	eof	\$ 0 List 1	List	accept

Parsing "()"

Here, peeling () off the stack reveals s_0 .
 $\text{Goto}(s_0, \text{Pair})$ is s_2 .

0	Goal	→	List
1	List	→	List Pair
2			Pair
3	Pair	→	(Pair)
4			()



The Parentheses Language

State	L'ahead	Stack	Handle	Action
—	(	\$ 0	—none—	—
0	(	\$ 0	—none—	shift 3
3	(	\$ 0 (3	—none—	shift 6
6	)	\$ 0 (3 (6	—none—	shift 10
10	)	\$ 0 (3 (6) 10	()	reduce 4
5	)	\$ 0 (3 Pair 5	—none—	shift 8
8	(	\$ 0 (3 Pair 5) 8	(Pair)	reduce 3
2	(	Here, peeling () off the stack reveals s_3, which represents the left context of an unmatched (. $Goto(s_3, Pair)$ is s_5, a state in which we expect a). That path leads to a reduction by production 3, $Pair \rightarrow (Pair)$.		
1	(			
3	)			
7	eof			
4	eof			
1	eof	\$ 0 List 1	List	accept

Parsing
"(()) ()"

Goal	→	List
List	→	List Pair
		Pair
3	Pair	→ (Pair)
4		()



The Parentheses Language

State	L'ahead	Stack	Handle	Action
—	(	\$ 0	—none—	—
0	(	\$ 0	—none—	shift 3
3				shift 6
6				shift 10
10				reduce 4
5				shift 8
8				reduce 3
2	(	\$ 0 Pair 2	Pair	reduce 2
1	(	\$ 0 List 1	—none—	shift 3
3	)	\$ 0 List 1 (3	—none—	shift 7
7	eof	\$ 0 List 1 (3) 7	()	reduce 1
4	eof	\$ 0 List 1 Pair 4	List Pair	reduce 0
1	eof	\$ 0 List 1	List	accept

Here, peeling () off the stack reveals s_1 , which represents the left context of a previously recognized List.
 Goto(s_1 , Pair) is s_4 , a state in which we can reduce List Pair to List (on lookahead of either (or eof).

Parsing
 "(()) ()"

Three copies of
 "reduce 4" with
 different context
 — produce three
 distinct behaviors

0	Goal	→	List
1	List	→	List Pair
2			Pair
3	Pair	→	(Pair)
4			()



LR(1) Parsers

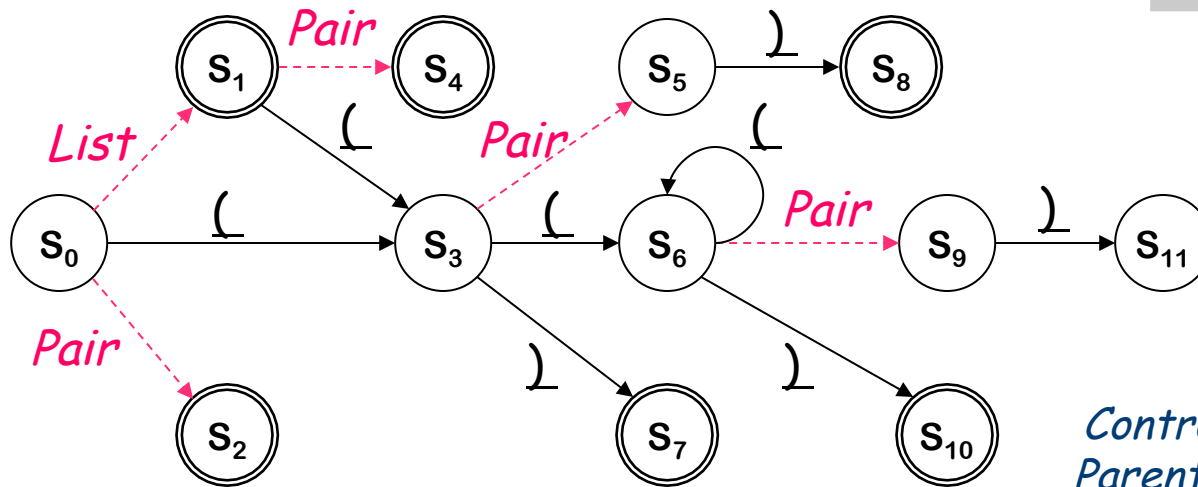
How does this LR(1) stuff work?

- Unambiguous grammar $\Rightarrow$ unique rightmost derivation
- Keep upper fringe on a stack
 - All active handles include top of stack (TOS)
 - Shift inputs until TOS is right end of a handle
- Language of handles is regular (finite)
 - Build a handle-recognizing DFA to control the stack-based recognizer
 - ACTION & GOTO tables encode the DFA
- To match a subterm, invoke the DFA recursively
 - leave old DFA's state on stack and go on
- Final state in DFA $\Rightarrow$ a *reduce* action
 - Pop rhs off the stack to reveal invoking state
 - $\rightarrow$ "It would be legal to recognize an x , and we did ..."
 - New state is GOTO[revealed state, *lhs*]
 - Take a DFA transition on the new *NT* — the lhs we just pushed...

LR(1) Parsers

The Control DFA for the Parentheses Language

0	Goal	→	List
1	List	→	List Pair
2			Pair
3	Pair	→	(Pair)
4			()



Control DFA for the Parentheses Language

Transitions on terminals represent shift actions [ACTION]

Transitions on **nonterminals** follow reduce actions [GOTO]

The table construction derives this DFA from the grammar



Building LR(1) Tables

How do we generate the ACTION and GOTO tables?

- Use the grammar to build a model of the Control DFA
- Encode actions & transitions in ACTION & GOTO tables
- If construction succeeds, the grammar is LR(1)
 - “Succeeds” means defines each table entry uniquely

The Big Picture

- Model the state of the parser
- Use two functions $goto(s, X)$ and $closure(s)$
 - $goto()$ is analogous to $move()$ in the subset construction
 - $closure()$ adds information to round out a state
- Build up the states and transition functions of the DFA
- Use this information to fill in the ACTION and GOTO tables

grammar symbol,
T or NT

fixed-point algorithm,
similar to the subset construction



LR(1) Items

The LR(1) table construction algorithm represents a valid configuration of an LR(1) parser with a data structure called an LR(1) items

An LR(**1**) item is a pair $[P, \delta]$, where

P is a production $A \rightarrow \beta$ with a $\cdot$ at some position in the *rhs*

δ is a single symbol lookahead (symbol $\equiv$ word or EOF)

The $\cdot$ in an item indicates the position of the top of the stack

$[A \rightarrow \cdot \beta \gamma, \underline{a}]$ means that the input seen so far is consistent with the use of $A \rightarrow \beta \gamma$ immediately after the symbol on top of the stack

$[A \rightarrow \beta \cdot \gamma, \underline{a}]$ means that the input seen so far is consistent with the use of $A \rightarrow \beta \gamma$ at this point in the parse, and that the parser has already recognized β (that is, β is on top of the stack).

$[A \rightarrow \beta \gamma \cdot, \underline{a}]$ means that the parser has seen $\beta \gamma$, and that a lookahead symbol of $\underline{a}$ is consistent with reducing to A .



LR(1) Items

The production $A \rightarrow \beta$, where $\beta = B_1 B_2 B_3$ with lookahead $\underline{a}$, can give rise to 4 items

$[A \rightarrow \cdot B_1 B_2 B_3, \underline{a}]$, $[A \rightarrow B_1 \cdot B_2 B_3, \underline{a}]$, $[A \rightarrow B_1 B_2 \cdot B_3, \underline{a}]$, & $[A \rightarrow B_1 B_2 B_3 \cdot, \underline{a}]$

The set of LR(1) items for a grammar is **finite**

What's the point of all these lookahead symbols?

- Carry them along to help choose the correct reduction
- Lookaheads are bookkeeping, unless item has $\cdot$ at right end
 - Has no direct use in $[A \rightarrow \beta \cdot \gamma, \underline{a}]$
 - In $[A \rightarrow \beta \cdot, \underline{a}]$, a lookahead of $\underline{a}$ implies a reduction by $A \rightarrow \beta$
 - For $\{ [A \rightarrow \beta \cdot, \underline{a}], [B \rightarrow \gamma \cdot \delta, \underline{b}] \}$, $\underline{a} \Rightarrow \text{reduce to } A$; $\text{FIRST}(\delta) \Rightarrow \text{shift}$

$\Rightarrow$ Limited right context is enough to pick the actions



LR(1) Table Construction

High-level overview

- 1 Build the canonical collection of sets of LR(1) Items, I
 - a Begin in an appropriate state, s_0
 - ◆ $[S' \rightarrow \cdot S, \text{EOF}]$, along with any equivalent items
 - ◆ Derive equivalent items as $\text{closure}(s_0)$
 - b Repeatedly compute, for each s_k , and each X , $\text{goto}(s_k, X)$
 - ◆ If the set is not already in the collection, add it
 - ◆ Record all the transitions created by $\text{goto}()$
- This eventually reaches a fixed point

- 2 Fill in the table from the collection of sets of LR(1) items

The states of the canonical collection are precisely the states of the Control DFA

The construction traces the DFA's transitions



Computing Closures

$Closure(s)$ adds all the items implied by items already in s

- Any item $[A \rightarrow \beta \cdot B \delta, \underline{a}]$ implies $[B \rightarrow \cdot \tau, x]$ for each production with B on the *lhs*, and each $x \in FIRST(\delta \underline{a})$
- Since $\beta B \delta$ is valid, any way to derive $\beta B \delta$ is valid, too

The algorithm

```
Closure(s)
while (s is still changing)
   $\forall$  items  $[A \rightarrow \beta \cdot B \delta, \underline{a}] \in s$ 
     $\forall$  productions  $B \rightarrow \tau \in P$ 
       $\forall \underline{b} \in FIRST(\delta \underline{a})$  //  $\delta$  might be  $\epsilon$ 
        if  $[B \rightarrow \cdot \tau, \underline{b}] \notin s$ 
          then  $s \leftarrow s \cup \{[B \rightarrow \cdot \tau, \underline{b}]\}$ 
```

- Classic fixed-point method
- Halts because $s \subset ITEMS$
- Worklist version is faster
- *Closure "fills out" a state*

Lookaheads are
generated here



Example From SheepNoise

Initial step builds the item $[Goal \rightarrow \bullet SheepNoise, EOF]$ and takes its *closure*()

Closure($[Goal \rightarrow \bullet SheepNoise, EOF]$)

Item	Source
$[Goal \rightarrow \bullet SheepNoise, \underline{EOF}]$	Original item
$[SheepNoise \rightarrow \bullet SheepNoise \underline{baa}, \underline{EOF}]$	1, $\delta \underline{a}$ is <u>EOF</u>
$[SheepNoise \rightarrow \bullet \underline{baa}, \underline{EOF}]$	1, $\delta \underline{a}$ is <u>EOF</u>
$[SheepNoise \rightarrow \bullet SheepNoise \underline{baa}, \underline{baa}]$	2, $\delta \underline{a}$ is <u>baa EOF</u>
$[SheepNoise \rightarrow \bullet \underline{baa}, \underline{baa}]$	2, $\delta \underline{a}$ is <u>baa EOF</u>

Remember, this is the left-recursive SheepNoise; EaC shows the right-recursive version.

So, S_0 is

{ $[Goal \rightarrow \bullet SheepNoise, \underline{EOF}]$, $[SheepNoise \rightarrow \bullet SheepNoise \underline{baa}, \underline{EOF}]$,
 $[SheepNoise \rightarrow \bullet \underline{baa}, \underline{EOF}]$, $[SheepNoise \rightarrow \bullet SheepNoise \underline{baa}, \underline{baa}]$,
 $[SheepNoise \rightarrow \bullet \underline{baa}, \underline{baa}]$ }

0	Goal	→	SheepNoise
1	SheepNoise	→	SheepNoise <u>baa</u>
2			<u>baa</u>



Computing Gotos

$Goto(s, x)$ computes the state that the parser would reach if it recognized an x while in state s

- $Goto(\{ [A \rightarrow \beta \cdot X \delta, \underline{a}] \}, X)$ produces $[A \rightarrow \beta X \cdot \delta, \underline{a}]$ (obviously)
- It finds all such items & uses $closure()$ to fill out the state

The algorithm

```
 $Goto(s, X)$   
   $new \leftarrow \emptyset$   
   $\forall \text{ items } [A \rightarrow \beta \cdot X \delta, \underline{a}] \in s$   
     $new \leftarrow new \cup \{ [A \rightarrow \beta X \cdot \delta, \underline{a}] \}$   
  return  $closure(new)$ 
```

- Not a fixed-point method!
- Straightforward computation
- Uses $closure()$
- $Goto()$ moves us forward

Example from SheepNoise



S_0 is $\{ [Goal \rightarrow \cdot SheepNoise, \underline{EOF}], [SheepNoise \rightarrow \cdot SheepNoise \underline{baa}, \underline{EOF}], [SheepNoise \rightarrow \cdot \underline{baa}, \underline{EOF}], [SheepNoise \rightarrow \cdot SheepNoise \underline{baa}, \underline{baa}], [SheepNoise \rightarrow \cdot \underline{baa}, \underline{baa}] \}$

From Slide 16

$Goto(S_0, \underline{baa})$

- Loop produces

Item	Source
$[SheepNoise \rightarrow \underline{baa} \cdot, \underline{EOF}]$	Item 3 in s_0
$[SheepNoise \rightarrow \underline{baa} \cdot, \underline{baa}]$	Item 5 in s_0

- Closure adds nothing since $\cdot$ is at end of *rhs* in each item

In the construction, this produces s_2

$\{ [SheepNoise \rightarrow \underline{baa} \cdot, \{ \underline{EOF}, \underline{baa} \}] \}$

New, but *obvious*, notation for two distinct items

$[SheepNoise \rightarrow \underline{baa} \cdot, \underline{EOF}]$ &
 $[SheepNoise \rightarrow \underline{baa} \cdot, \underline{baa}]$

0	Goal	→	SheepNoise	
1	SheepNoise	→	SheepNoise <u>baa</u>	
2			<u>baa</u>	28



Building the Canonical Collection

Start from $s_0 = \text{closure}([S' \rightarrow S, \underline{\text{EOF}}])$

Repeatedly construct new states, until all are found

The algorithm

```
 $s_0 \leftarrow \text{closure}([S' \rightarrow S, \underline{\text{EOF}}])$   
 $S \leftarrow \{s_0\}$   
 $k \leftarrow 1$   
while ( $S$  is still changing)  
   $\forall s_j \in S \text{ and } \forall x \in (T \cup NT)$   
     $s_k \leftarrow \text{goto}(s_j, x)$   
    record  $s_j \rightarrow s_k$  on  $x$   
  if  $s_k \notin S$  then  
     $S \leftarrow S \cup \{s_k\}$   
     $k \leftarrow k + 1$ 
```

- Fixed-point computation
- Loop adds to S
- $S \subseteq 2^{\text{ITEMS}}$, so S is finite
- *Worklist version is faster*



Example from SheepNoise

Starts with S_0

$$S_0 : \{ [Goal \rightarrow \cdot SheepNoise, \underline{EOF}], [SheepNoise \rightarrow \cdot SheepNoise \underline{baa}, \underline{EOF}], \\ [SheepNoise \rightarrow \cdot \underline{baa}, \underline{EOF}], [SheepNoise \rightarrow \cdot SheepNoise \underline{baa}, \underline{baa}], \\ [SheepNoise \rightarrow \cdot \underline{baa}, \underline{baa}] \}$$

Iteration 1 computes

$$S_1 = Goto(S_0, SheepNoise) = \\ \{ [Goal \rightarrow SheepNoise \cdot, \underline{EOF}], [SheepNoise \rightarrow SheepNoise \cdot \underline{baa}, \underline{EOF}], \\ [SheepNoise \rightarrow SheepNoise \cdot \underline{baa}, \underline{baa}] \}$$

$$S_2 = Goto(S_0, \underline{baa}) = \{ [SheepNoise \rightarrow \underline{baa} \cdot, \underline{EOF}], \\ [SheepNoise \rightarrow \underline{baa} \cdot, \underline{baa}] \}$$

Nothing more to compute, since $\cdot$ is at the end of every item in S_3 .

Iteration 2 computes

$$S_3 = Goto(S_1, \underline{baa}) = \{ [SheepNoise \rightarrow SheepNoise \underline{baa} \cdot, \underline{EOF}], \\ [SheepNoise \rightarrow SheepNoise \underline{baa} \cdot, \underline{baa}] \}$$

0	Goal	→	SheepNoise
1	SheepNoise	→	SheepNoise <u>baa</u>
2			<u>baa</u>

Example from SheepNoise



$S_0 : \{ [Goal \rightarrow \cdot SheepNoise, \underline{EOF}], [SheepNoise \rightarrow \cdot SheepNoise \underline{baa}, \underline{EOF}],$
 $[SheepNoise \rightarrow \cdot \underline{baa}, \underline{EOF}], [SheepNoise \rightarrow \cdot SheepNoise \underline{baa}, \underline{baa}],$
 $[SheepNoise \rightarrow \cdot \underline{baa}, \underline{baa}] \}$

$S_1 = Goto(S_0, SheepNoise) =$
 $\{ [Goal \rightarrow SheepNoise \cdot, \underline{EOF}], [SheepNoise \rightarrow SheepNoise \cdot \underline{baa}, \underline{EOF}],$
 $[SheepNoise \rightarrow SheepNoise \cdot \underline{baa}, \underline{baa}] \}$

$S_2 = Goto(S_0, \underline{baa}) = \{ [SheepNoise \rightarrow \underline{baa} \cdot, \underline{EOF}],$
 $[SheepNoise \rightarrow \underline{baa} \cdot, \underline{baa}] \}$

$S_3 = Goto(S_1, \underline{baa}) = \{ [SheepNoise \rightarrow SheepNoise \underline{baa} \cdot, \underline{EOF}],$
 $[SheepNoise \rightarrow SheepNoise \underline{baa} \cdot, \underline{baa}] \}$

0	Goal	→	SheepNoise
1	SheepNoise	→	SheepNoise <u>baa</u>
2			<u>baa</u>



Filling in the ACTION and GOTO Tables

The algorithm

x is the state number

$\forall$ set $S_x \in S$

$\forall$ item $i \in S_x$

if i is $[A \rightarrow \beta \cdot \underline{a} \delta, \underline{b}]$ and $\text{goto}(S_x, \underline{a}) = S_k, \underline{a} \in T$

then $\text{ACTION}[x, \underline{a}] \leftarrow \text{"shift } k\text{"}$

• before $T \Rightarrow \text{shift}$

else if i is $[S' \rightarrow S \cdot, \underline{\text{EOF}}]$

then $\text{ACTION}[x, \underline{\text{EOF}}] \leftarrow \text{"accept"}$

have Goal $\Rightarrow \text{accept}$

else if i is $[A \rightarrow \beta \cdot, \underline{a}]$

then $\text{ACTION}[x, \underline{a}] \leftarrow \text{"reduce } A \rightarrow \beta\text{"}$

• at end $\Rightarrow \text{reduce}$

$\forall n \in NT$

if $\text{goto}(S_x, n) = S_k$

then $\text{GOTO}[x, n] \leftarrow k$

Many items generate no table entry

$\rightarrow \text{Closure}()$ instantiates $\text{FIRST}(X)$ directly for $[A \rightarrow \beta \cdot X \delta, \underline{a}]$



Example from SheepNoise

$S_0 : \{ [Goal \rightarrow \cdot SheepNoise, \underline{EOF}], [SheepNoise \rightarrow \cdot SheepNoise \underline{baa}, \underline{EOF}],$
 $[SheepNoise \rightarrow \cdot \underline{baa} \underline{EOF}], [SheepNoise \rightarrow \cdot SheepNoise \underline{baa}, \underline{baa}],$
 $[SheepNoise \rightarrow \cdot \underline{baa}, \underline{baa}] \}$

• before $T \Rightarrow \text{shift } k$

$S_1 = Goto(S_0, SheepNoise) =$

$\{ [Goal \rightarrow SheepNoise \cdot, \underline{EOF}], [SheepNoise \rightarrow SheepNoise \cdot \underline{baa}, \underline{EOF}],$
 $[SheepNoise \rightarrow SheepNoise \cdot \underline{baa}, \underline{baa}] \}$

$S_2 = Goto(S_0, \underline{baa}) = \{ [SheepNoise \rightarrow \underline{baa} \cdot, \underline{EOF}],$
 $[SheepNoise \rightarrow \underline{baa} \cdot, \underline{baa}] \}$

so, $ACTION[s_0, \underline{baa}]$ is
"shift S_2 " (clause 1)
(items define same entry)

$S_3 = Goto(S_1, \underline{baa}) = \{ [SheepNoise \rightarrow SheepNoise \underline{baa} \cdot, \underline{EOF}],$
 $[SheepNoise \rightarrow SheepNoise \underline{baa} \cdot, \underline{baa}] \}$



Example from SheepNoise

$S_0 : \{ [Goal \rightarrow \cdot SheepNoise, \underline{EOF}], [SheepNoise \rightarrow \cdot SheepNoise \underline{baa}, \underline{EOF}],$
 $[SheepNoise \rightarrow \cdot \underline{baa}, \underline{EOF}], [SheepNoise \rightarrow \cdot SheepNoise \underline{baa}, \underline{baa}],$
 $[SheepNoise \rightarrow \cdot \underline{baa}, \underline{baa}] \}$

$S_1 = Goto(S_0, SheepNoise) =$

$\{ [Goal \rightarrow SheepNoise \cdot, \underline{EOF}], [SheepNoise \rightarrow SheepNoise \cdot \underline{baa}, \underline{EOF}],$
 $[SheepNoise \rightarrow SheepNoise \cdot \underline{baa}, \underline{baa}] \}$

$S_2 = Goto(S_0, \underline{baa}) = \{ [SheepNoise \rightarrow \underline{baa} \cdot, \underline{EOF}],$
 $[SheepNoise \rightarrow \underline{baa} \cdot, \underline{baa}] \}$

so, ACTION[$S_1, \underline{baa}$]
is "shift S_3 " (clause 1)

$S_3 = Goto(S_1, \underline{baa}) = \{ [SheepNoise \rightarrow SheepNoise \underline{baa} \cdot, \underline{EOF}],$
 $[SheepNoise \rightarrow SheepNoise \underline{baa} \cdot, \underline{baa}] \}$



Example from SheepNoise

$S_0 : \{ [Goal \rightarrow \cdot SheepNoise, \underline{EOF}], [SheepNoise \rightarrow \cdot SheepNoise \underline{baa}, \underline{EOF}],$
 $[SheepNoise \rightarrow \cdot \underline{baa}, \underline{EOF}], [SheepNoise \rightarrow \cdot SheepNoise \underline{baa}, \underline{baa}],$
 $[SheepNoise \rightarrow \cdot \underline{baa}, \underline{baa}] \}$

$S_1 = Goto(S_0, SheepNoise) =$
 $\{ [Goal \rightarrow SheepNoise \cdot, \underline{EOF}], [SheepNoise \rightarrow SheepNoise \cdot \underline{baa}, \underline{EOF}],$
 $[SheepNoise \rightarrow SheepNoise \cdot \underline{baa}, \underline{baa}] \}$

so, ACTION[S_1, EOF]
is "accept" (clause 2)

$S_2 = Goto(S_0, \underline{baa}) = \{ [SheepNoise \rightarrow \underline{baa} \cdot, \underline{EOF}],$
 $[SheepNoise \rightarrow \underline{baa} \cdot, \underline{baa}] \}$

$S_3 = Goto(S_1, \underline{baa}) = \{ [SheepNoise \rightarrow SheepNoise \underline{baa} \cdot, \underline{EOF}],$
 $[SheepNoise \rightarrow SheepNoise \underline{baa} \cdot, \underline{baa}] \}$

Example from SheepNoise



$S_0 : \{ [Goal \rightarrow \cdot SheepNoise, \underline{EOF}], [SheepNoise \rightarrow \cdot SheepNoise \underline{baa}, \underline{EOF}],$
 $[SheepNoise \rightarrow \cdot \underline{baa}, \underline{EOF}], [SheepNoise \rightarrow \cdot SheepNoise \underline{baa}, \underline{baa}],$
 $[SheepNoise \rightarrow \cdot \underline{baa}, \underline{baa}] \}$

$S_1 = Goto(S_0, SheepNoise) =$
 $\{ [Goal \rightarrow SheepNoise \cdot, \underline{EOF}], [SheepNoise \rightarrow SheepNoise \cdot \underline{baa}, \underline{EOF}],$
 $[SheepNoise \rightarrow SheepNoise \cdot \underline{baa}, \underline{baa}] \}$

$S_2 = Goto(S_0, \underline{baa}) = \{ [SheepNoise \rightarrow \underline{baa} \cdot, \underline{EOF}],$
 $[SheepNoise \rightarrow \underline{baa} \cdot, \underline{baa}] \}$

so, ACTION[S_2 , EOF] is
"reduce 3" (clause 3)

$S_3 = Goto(S_1, \underline{baa}) = \{ [SheepNoise \rightarrow SheepNoise \underline{baa} \cdot, \underline{EOF}],$
 $[SheepNoise \rightarrow SheepNoise \underline{baa} \cdot, \underline{baa}] \}$



Example from SheepNoise

$S_0 : \{ [Goal \rightarrow \cdot SheepNoise, \underline{EOF}], [SheepNoise \rightarrow \cdot SheepNoise \underline{baa}, \underline{EOF}], [SheepNoise \rightarrow \cdot \underline{baa}, \underline{EOF}], [SheepNoise \rightarrow \cdot SheepNoise \underline{baa}, \underline{baa}], [SheepNoise \rightarrow \cdot \underline{baa}, \underline{baa}] \}$

$ACTION[S_3, \underline{EOF}]$ is
"reduce 2" (clause 3)

$S_1 = Goto(S_0, \underline{baa}) = \{ [SheepNoise \rightarrow \cdot, \underline{EOF}], [SheepNoise \rightarrow SheepNoise \cdot \underline{baa}, \underline{EOF}], [SheepNoise \rightarrow SheepNoise \cdot \underline{baa}, \underline{baa}] \}$

$ACTION[S_2, \underline{baa}]$ is
"reduce 3" (clause 3)

$S_2 = Goto(S_0, \underline{baa}) = \{ [SheepNoise \rightarrow \underline{baa} \cdot, \underline{EOF}], [SheepNoise \rightarrow \underline{baa} \cdot, \underline{baa}] \}$

$S_3 = Goto(S_1, \underline{baa}) = \{ [SheepNoise \rightarrow SheepNoise \underline{baa} \cdot, \underline{EOF}], [SheepNoise \rightarrow SheepNoise \underline{baa} \cdot, \underline{baa}] \}$

$ACTION[S_3, \underline{baa}]$ is
"reduce 2", as well

$ACTION[S_2, \underline{EOF}]$ is
"reduce 2" (clause 3)



Example from SheepNoise

The *GOTO* Table records *Goto* transitions on NTs

$S_0 : \{ [Goal \rightarrow \cdot SheepNoise, \underline{EOF}], [SheepNoise \rightarrow \cdot SheepNoise \underline{baa}, \underline{EOF}],$
 $[SheepNoise \rightarrow \cdot \underline{baa}, \underline{EOF}], [SheepNoise \rightarrow \cdot SheepNoise \underline{baa}, \underline{baa}],$
 $[SheepNoise \rightarrow \cdot \underline{baa}, \underline{baa}] \}$

$S_1 = Goto(S_0, SheepNoise) =$
 $\{ [Goal \rightarrow SheepNoise \cdot, \underline{EOF}], [SheepNoise \rightarrow SheepNoise \cdot \underline{baa}, \underline{EOF}],$
 $[SheepNoise \rightarrow SheepNoise \cdot \underline{baa}, \underline{baa}] \}$

$S_2 = Goto(S_0, \underline{baa}) = \{ [SheepNoise \rightarrow \underline{baa} \cdot, \underline{EOF}],$
 $[SheepNoise \rightarrow \underline{baa} \cdot, \underline{baa}] \}$

$S_3 = Goto(S_1, \underline{baa}) = \{ [SheepNoise \rightarrow SheepNoise \underline{baa} \cdot, \underline{EOF}],$
 $[SheepNoise \rightarrow SheepNoise \underline{baa} \cdot, \underline{baa}] \}$

Based on T, not NT

Only 1 transition in the
entire *GOTO* table

Remember, we recorded these so
we don't need to recompute them.



ACTION & GOTO Tables

Here are the tables for the augmented left-recursive *SheepNoise* grammar

The tables

ACTION TABLE		
State	EOF	<u>baa</u>
0	—	<i>shift 2</i>
1	<i>accept</i>	<i>shift 3</i>
2	<i>reduce 2</i>	<i>reduce 2</i>
3	<i>reduce 1</i>	<i>reduce 1</i>

GOTO TABLE	
State	<i>SheepNoise</i>
0	1
1	0
2	0
3	0

Remember, this is the left-recursive *SheepNoise*; EaC shows the right-recursive version.

The grammar

0	<i>Goal</i>	→	<i>SheepNoise</i>
1	<i>SheepNoise</i>	→	<i>SheepNoise</i> <u><i>baa</i></u>
2			<u><i>baa</i></u>



What can go wrong?

What if set s contains $[A \rightarrow \beta \cdot \underline{a} \gamma, \underline{b}]$ and $[B \rightarrow \beta \cdot, \underline{a}]$?

- First item generates "shift", second generates "reduce"
- Both define $ACTION[s, \underline{a}]$ — cannot do both actions
- This is a fundamental ambiguity, called a *shift/reduce error*
- Modify the grammar to eliminate it (if-then-else)
- Shifting will often resolve it correctly

What if set s contains $[A \rightarrow \gamma \cdot, \underline{a}]$ and $[B \rightarrow \gamma \cdot, \underline{a}]$?

- Each generates "reduce", but with a different production
- Both define $ACTION[s, \underline{a}]$ — cannot do both reductions
- This is a fundamental ambiguity, called a *reduce/reduce conflict*
- Modify the grammar to eliminate it (PL/I's overloading of (...))

In either case, the grammar is not LR(1)



Shrinking the Tables

Three options:

- Combine terminals such as number & identifier, + & -, * & /
 - Directly removes a column, may remove a row
 - For expression grammar, 198 (vs. 384) table entries
- Combine rows or columns
 - Implement identical rows once & remap states
 - Requires extra indirection on each lookup
 - Use separate mapping for ACTION & for GOTO
- Use another construction algorithm
 - Both LALR(1) and SLR(1) produce smaller tables
 - Implementations are readily available

left-recursive expression grammar with precedence, see §3.7.2 in EAC

classic space-time tradeoff

Is handle recognizing DFA minimal?



LR(k) versus LL(k)

Finding Reductions

LR(k) $\Rightarrow$ Each reduction in the parse is detectable with

- $\rightarrow$ the complete left context,
- $\rightarrow$ the reducible phrase, itself, and
- $\rightarrow$ the k terminal symbols to its right

generalizations of
LR(1) and LL(1) to
longer lookaheads

LL(k) $\Rightarrow$ Parser must select the reduction based on

- $\rightarrow$ The complete left context
- $\rightarrow$ The next k terminals

Thus, LR(k) examines more context

"... in practice, programming languages do not actually seem to fall in the gap between LL(1) languages and deterministic languages"

J.J. Horning, "LR Grammars and Analysers", in Compiler Construction, An Advanced Course, Springer-Verlag, 1976



LL(1) versus LR(1)

The following LR(1) grammar has no LL(1) counterpart[†]

0	<i>Start</i>	→	<i>A</i>
1			<i>B</i>
2	<i>A</i>	→	(<i>A</i>)
3			<u><i>a</i></u>
4	<i>B</i>	→	(<i>B</i> }
5			<u><i>a</i></u>

- A top-down parser cannot choose between *A* and *B* without arbitrary lookahead.
- A bottom-up LR parser simply shifts the words until it finds a *)* or a *}*, then makes the right reduction.

The important question, however, is whether or not this distinction is meaningful.

- Is there some programming language construct where this distinction matters?
- Does it buy us additional expressiveness?

[†]More precisely, the language described by this LR(1) grammar cannot be described with an LL(1) grammar. In fact, the language has no LL(*k*) grammar, for finite *k*.

Action & Goto Tables for Waite's Example



	EOF	(	)	<u>a</u>	}
s ₀		s 4		s 5	
s ₁	acc				
s ₂	r 2				
s ₃	r 3				
s ₄		s 8		s 9	
s ₅	r 5				
s ₆			s 10		
s ₇					s 11
s ₈		s 8		s 9	
s ₉			r 5		r 7
s ₁₀	r 4				
s ₁₁	r 6				
s ₁₂			s 14		
s ₁₃					s 15
s ₁₄			r 4		
s ₁₅					r 6

	S	A	B
s ₀	1	2	3
s ₁			
s ₂			
s ₃			
s ₄		6	7
s ₅			
s ₆			
s ₇			
s ₈		12	13
s ₉			
s ₁₀			
s ₁₁			
s ₁₂			
s ₁₃			
s ₁₄			
s ₁₅			

0	Start	→	A
1			B
2	A	→	(A)
3			<u>a</u>
4	B	→	(B }
5			<u>a</u>



LR(1) versus LL(1)

Other Non-LL Grammars

0	$B \rightarrow R$
1	$ (B)$
2	$R \rightarrow E = E$
3	$E \rightarrow \underline{a}$
4	$ \underline{b}$
5	$ (E + E)$

Example from D.E Knuth, "Top-Down Syntactic Analysis," *Acta Informatica*, 1:2 (1971), pages 79-110

This grammar is actually LR(0)

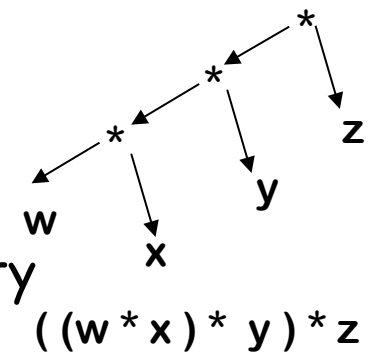
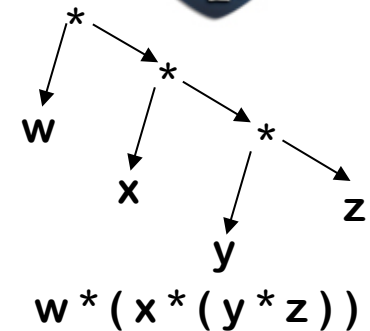
0	$S \rightarrow \underline{a} A \underline{b}$
1	$ \underline{c}$
2	$A \rightarrow \underline{b} S$
3	$ B \underline{b}$
4	$B \underline{a} A$
5	$ \underline{c}$

Example from Lewis, Rosenkrantz, & Stearns book, "Compiler Design Theory," (1976), Figure 13.1

Left Recursion versus Right Recursion



- Right recursion
 - Required for termination in top-down parsers
 - Uses (on average) more stack space
 - Naïve right recursion produces right-associativity
- Left recursion
 - Works fine in bottom-up parsers
 - Limits required stack space
 - Naïve left recursion produces left-associativity
- Rule of thumb
 - Left recursion for bottom-up parsers
 - Right recursion for top-down parsers

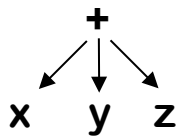




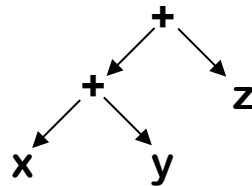
Associativity

What difference does it make?

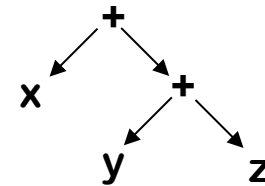
- Can change answers in floating-point arithmetic
- Can change opportunities for optimization
- Consider $x+y+z$



*Ideal
operator*



*Left
association*



*Right
association*

What if $y+z$ occurs elsewhere? Or $x+y$? or $x+z$?

The compiler may want to change the "shape" of expressions

- What if $x = 2$ & $z = 17$? Neither left nor right exposes 19.
- Best shape is function of surrounding context (see Comp 512)



Error Detection and Recovery

Error Detection

- Recursive descent
 - Parser takes the last else clause in a routine
 - Compiler writer can code almost any arbitrary action
- Table-driven LL(1)
 - In state s_i facing word x , entry is an error
 - Report the error, valid entries in row for s_i encode possibilities
- Table-driven LR(1)
 - In state s_i facing word x , entry is an error
 - Report the error, shift states in row encode possibilities
 - Can precompute better messages from LR(1) items



Error Detection and Recovery

Error Recovery

- Table-driven LL(1)
 - Treat as missing token, e.g. '}', $\Rightarrow$ expand by desired symbol
 - Treat as extra token, e.g., 'x - + y', $\Rightarrow$ pop stack and move ahead
- Table-driven LR(1)
 - Treat as missing token, e.g. '}', $\Rightarrow$ shift the token
 - Treat as extra token, e.g., 'x - + y', $\Rightarrow$ don't shift the token

Can pre-compute sets of states
with appropriate entries...



Error Detection and Recovery

One common strategy is “hard token” recovery

Skip ahead in input until we find some “hard” token, e.g. ‘;’

- ‘;’ separates statements, makes a logical break in the parse
- Resynchronize state, stack, and input to point after hard token
 - LL(1): pop stack until we find a row with entry for ‘;’
 - LR(1): pop stack until we find a state with a reduction on ‘;’
- Does not correct the input, rather it allows parse to proceed

```
NT ← pop()
repeat until Table[NT,‘;’] ≠ error
  NT ← pop()
token ← NextToken()
repeat until token = ‘;’
  token ← NextToken()
```

Resynchronizing an LL(1) parser

```
repeat until token = ‘;’
  shift token
  shift  $s_e$ 
  token ← NextToken()
reduce by error production
// pops all that state off stack
```

Resynchronizing an LR(1) parser



Shrinking the ACTION and GOTO Tables

Three options:

- Combine terminals such as number & identifier, + & -, * & /
 - Directly removes a column, may remove a row
 - For expression grammar, 198 (vs. 384) table entries
- Combine rows or columns
 - Implement identical rows once & remap states
 - Requires extra indirection on each lookup
 - Use separate mapping for ACTION & for GOTO
- Use another construction algorithm
 - Both LALR(1) and SLR(1) produce smaller tables
 - LALR(1) represents each state with its "core" items
 - SLR(1) uses LR(0) items and the FOLLOW set
 - Implementations are readily available

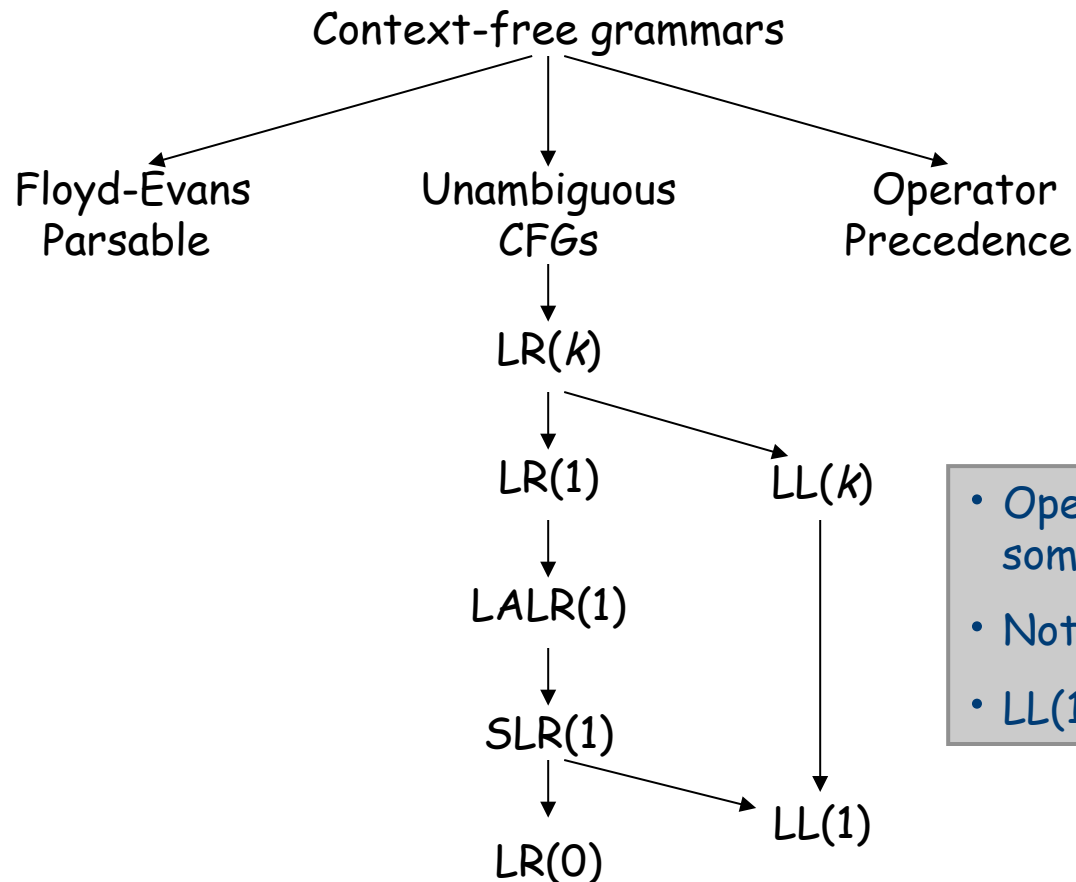
left-recursive expression grammar with precedence, see §3.6.2 in EAC

classic space-time tradeoff

Fewer grammars, same languages



Hierarchy of Context-Free Grammars



- Operator precedence includes some ambiguous grammars
- Note sub-categories of LR(1)
- LL(1) is a subset of SLR(1)

*The inclusion hierarchy for
context-free grammars*

Summary



	<i>Advantages</i>	<i>Disadvantages</i>
<i>Top-down Recursive descent, LL(1)</i>	Fast Good locality Simplicity Good error detection	Hand-coded High maintenance Right associativity
<i>LR(1)</i>	Fast Deterministic langs. Automatable Left associativity	Large working sets Poor error messages Large table sizes